WEEK 3 – DATA 5600

Spring 2026

Pol Monné Parera (L30170743)
pmonneparera@lewisu.edu

1. Technical Description

1.1 Long Short-Term Memory (LSTM) Neural Network

Long Short-Term Memory (LSTM) is a specialized type of Recurrent Neural Network (RNN) architecture. They are designed to learn long term dependencies in sequential data and overcome the vanishing gradient problem (an issue that limits traditional RNNs). Introduced in 1997 by Hochreiter and Schmidhuber, this type of neural networks is widely used for tasks like time-series forecasting (basis for this assignment), natural language processing and speech recognition.

1.2 Dropout Regularization

Dropout regularization sets a predetermined number of neuron activations to zero during each traying pass, preventing co-adoption. Meaning that neurons are unable to rely on their neighbors, making each learn independent useful features.

1.3 Adam Optimizer

Adam (Adaptive Moment Estimation) is a widely used optimization algorithm in deep learning, maintaining per-parameter learning rates by tracking the exponential moving average of both the gradient and the squared gradient.

It is applied to this assignment as the learning rate adapts per parameter and it handles sparse or noisy gradients robustly, both being common in stock data and making this optimizer useful for time-series financial data.

1.4 Mean Squared Error Loss

Mean Squared Error (MSE) is the regression loss. By penalizing large errors quadratically, it becomes desirable in price prediction where large mispredictions are disproportionately costly. The model implemented tries to minimize MSE over the training set during backpropagation time.

1.5 Vanishing Gradient Problem

The vanishing gradient problem is a major training obstacle in deep neural networks and RNNs. There gradients are used to update model weights via backpropagation, becoming extremely small as they propagate backward. All of this results in earlier neurons to stop learning and consequently meaning slower convergence and lower performance.

2. Design of the Algorithm

2.1 Dataset

The stock analyzed in this project corresponds to that of Apple Inc. (AAPL). Its daily stock data is fetched via Yahoo Finance using the yfinance Python library, covering January 2022 to December 2024. The five OHLCV attributes (Open, High, Low, Close, Volume) are used as features. Close at t + 1 is the prediction target.

2.2 Pseudocode

```
// Load data
GET stock data for AAPL from 2022 to 2024
KEEP only Open, High, Low, Close, Volume columns

// Create sequences (window size = 60)
FOR each time step from 60 to end:
        CREATE input sequence using previous 60 days
        SET target as Close price of current day

// Build model
CREATE neural network:
  LSTM layer (128 units)
  Dropout
  LSTM layer (64 units)
  Dropout
  Dense layer (32 units)
  Output layer (1 unit)

CONFIGURE model with optimizer and error function

// Train model
TRAIN model using training data
VALIDATE using validation data
USE early stopping to avoid overfitting

// Evaluate model
PREDICT values on test data
CONVERT predictions back to original scale
CALCULATE RMSE and MAE errors
```

2.2 Architecture Table

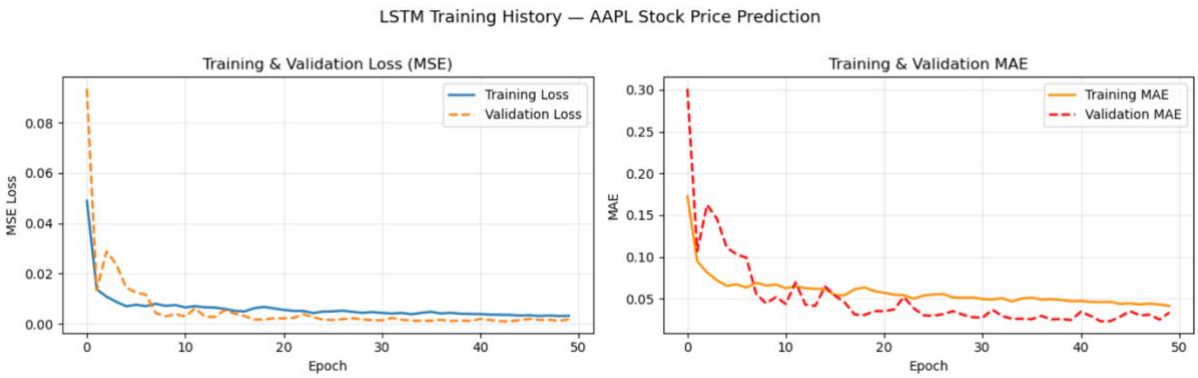
In the following table the number of layers and their units can be observed.

Layer	Units / Config
LSTM-1	128 units, return_sequences=True

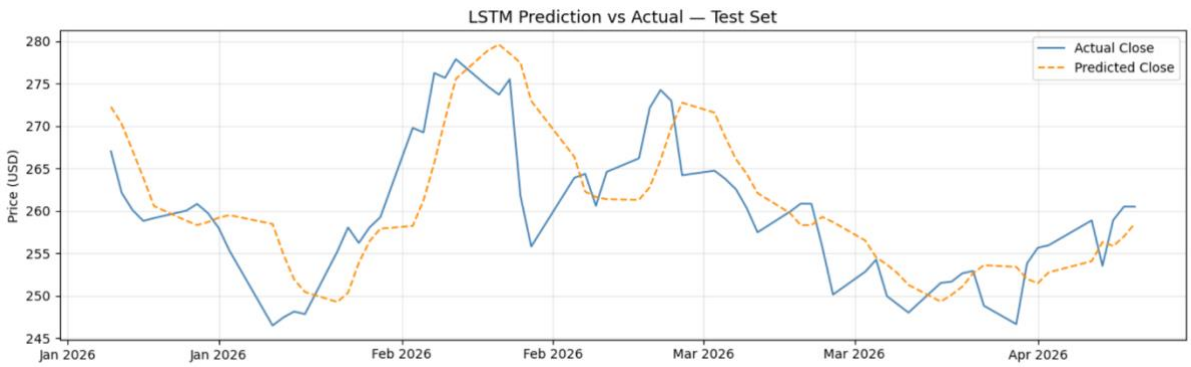
Dropout-1	rate = 0.2
LSTM-2	64 units, return_sequences=False
Dropout-2	rate = 0.2
Dense (hidden)	32 units, ReLU
Dense (output)	1 unit, Linear

3. Results of the Algorithm

3.1 Training and Validation Loss Curves



3.2 Prediction vs Actual



3.3 Evaluation Metrics

Metric	Test Set Value
RMSE (USD)	5.7124
MAE (USD)	4.5752
MSE	32.631679
MAPE (%)	1.76

4. Analysis of Results

4.1 Did I obtain what I expected?

It was expected that the model would have a general ability to track next-day direction. Even though financial price series are known to be low signal (day-to-day noise often dominates the learnable pattern), the LSTM was expected to outperform a univariate baseline since OHLCV data carries predictive information about next-day momentum.

4.2 Surprises

The most notable surprise consisted of how early the training and validation losses plateaued. Nevertheless, early stopping was not triggered, indicating that performance improvements the early stopping criterion was not sufficiently sensitive.

4.3 Conclusions

The LSTM was able to successfully learn to approximate the next-day closing prices by utilizing the 60-day windows it was given. Its memory made it well-suited for the task when comparing it to connected networks, which treat each day as an independent data point, losing temporal ordering. Fyrthermore, the results obtained were consistent with the Efficient Market Hypothesis, which states that the publicly available historical price and volume data contains limited exploitable information for predicting future price movements.

Some possible future improvements could include the following:

1. Adding some technical indicators like RSI, MACD or Bollinger Bands as additional input features.
2. Multi-step forecasting to predict a week ahead rather than a single day.
3. Ensemble methods to combine the LSTM with gradient-boosted trees.

5. Program Listings

```python
def create_sequences(features_scaled, close_scaled, look_back):
    """
    Build (X, y) pairs for next-day close prediction.
    X shape: (samples, look_back, n_features)
    y shape: (samples,)  — next-day scaled Close
    """
    X, y = [], []
    for i in range(look_back, len(features_scaled)):
        X.append(features_scaled[i - look_back : i])      # window of past N days
        y.append(close_scaled[i, 0])                      # next-day Close (scalar)
    return np.array(X), np.array(y)


X, y = create_sequences(scaled_all, scaled_close, LOOK_BACK)
```

Figure 1 Sequence Construction

```python
def build_model(look_back, n_features):
    """
    Stacked LSTM model for next-day stock close prediction.

    Architecture:
        LSTM(128) → Dropout(0.2) → LSTM(64) → Dropout(0.2) → Dense(32) → Dense(1)
    """
    model = Sequential([
        # Layer 1: 128-unit LSTM, returns full sequence for stacking
        LSTM(128, return_sequences=True,
             input_shape=(look_back, n_features),
             name='lstm_1'),
        Dropout(0.2, name='dropout_1'),

        # Layer 2: 64-unit LSTM, returns only the final hidden state
        LSTM(64, return_sequences=False, name='lstm_2'),
        Dropout(0.2, name='dropout_2'),

        # Fully connected layers
        Dense(32, activation='relu', name='dense_hidden'),
        Dense(1,  activation='linear', name='output'),
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='mse',
        metrics=['mae']
    )
    return model


model = build_model(look_back=LOOK_BACK, n_features=len(FEATURES))
model.summary()
```

Figure 2 Model Definition

```python
EPOCHS = 50
BATCH_SIZE = 32

early_stop = EarlyStopping(
    monitor = 'val_loss',
    patience = 10,
    restore_best_weights = True,
    verbose = 1
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    callbacks=[early_stop],
    verbose=1
)
```

Figure 3 Training

References

Dataset: Apple Inc. (AAPL) daily stock data. Yahoo Finance. Retrieved via yfinance Python library. https://finance.yahoo.com/quote/AAPL/history/